10 Forgetting Attention Policies:
Reference Implementations, Correctness Tests, and Scaling Behavior on an 18M-Class Transformer

Vuk Rosic

Independent Research Project

May 26, 2026

Abstract

We study whether simple forgetting and memory policies can improve causal attention when implemented as readable PyTorch reference modules. We compare dense attention against ten non-dense policies under a fixed one-GPU budget, sweeping context lengths from 2K to 8K on an 18M-class transformer. In this unfused setup, dense attention has the best loss and throughput. The strongest memory variants are the simplest ones: block-mean compression, age decay, hierarchical summaries, and surprise retention. This paper and code repository provides clearly defined and correctly implemented forgetting mechanisms and experiments ready for kernel writers or larger-scale follow-up experiments, as well as initial small-scale results.

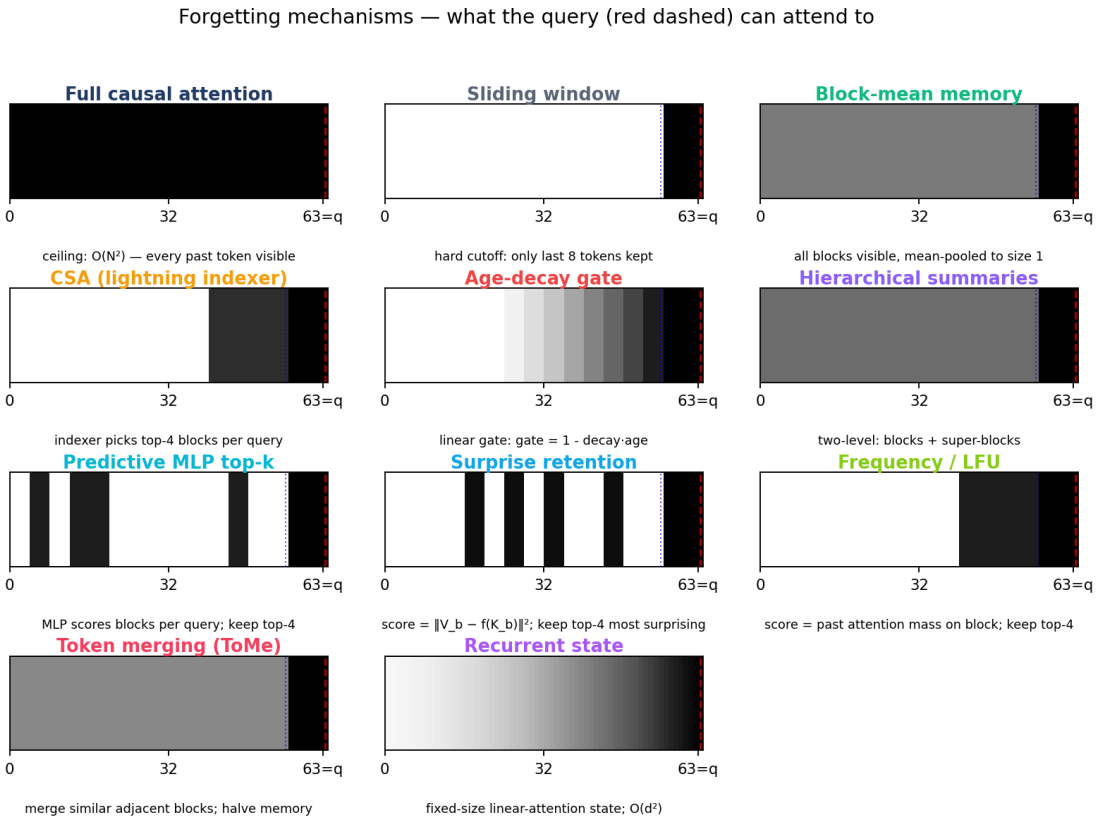


Figure 1: Memory illustration for the mechanisms in this paper. Black regions are fully visible to the query, gray regions are retained through memory/gating, and white regions are outside the visible memory.

1 Literature Review

Long-context transformers usually change one of four things: which tokens may be attended to, how past state is stored, how old information is retrieved, or how sequence length is reduced before attention runs. Local and sparse-attention

methods such as Longformer, BigBird, Reformer, and Routing Transformer reduce the number of keys each query compares against [6, 7, 8, 9]. Memory systems such as Transformer-XL and Compressive Transformer carry state across segments or compress old states that would otherwise leave the window [4, 5]. Retrieval methods such as kNN-LM and Memorizing Transformers externalize some of that memory into a datastore [11, 10].

Modern large models also care about memory compression. DeepSeek-V2 and DeepSeek-V3 use Multi-head Latent Attention to reduce KV-cache cost at scale [1, 2]. Our CSA baseline is a small reference point in that family: old context is still attention material, but it is represented more cheaply before attention reads it. Token-merging work such as ToMe motivates a different primitive: compress redundant content by merging, not only by masking or dropping [12].

2 Goal

- Provide a reference implementation of ten forgetting-attention policies ready to scale up by kernel writers and follow-up experiments.
- Explain each idea clearly with math, code, and a diagram.

3 Policies Compared

Each policy is an alternative per-layer attention module inside the same transformer block. Figure 1 shows what each mechanism's attention pattern looks like at the last query position.

- **dense** (classic baseline) — full causal SDPA with GQA and RoPE.
- **local** — sliding window of $W = 64$, no memory. Included as the isolated baseline so the other policies (which all contain local attention plus added memory) can be measured against pure local: does the extra memory machinery add value, or is local doing most of the work?
- **compressed_memory** — block-mean compression, all blocks visible, no gate.
- **csa** — lightning indexer scores compressed blocks; top $k = 4$ kept per query (DeepSeek-style compressed sparse attention baseline).
- **age_forgetting** — linear age-decay gate: gate $= 1 - r \cdot$ age on compressed blocks.
- **hierarchical** — two-level: blocks plus super-blocks-of-blocks, each level gated by $1/(\text{level} + 1)$.
- **predictive** — small MLP scores blocks per query from (Q, K_b, age); top-k kept.
- **surprise_retention** (new) — per-block, query-independent score $s_b = \|V_b - f(K_b)\|^2$ where f is a learned predictor; top-k most-surprising blocks kept.
- **frequency_lfu** (new) — score is the causal cumulative sum of past attention mass on each block: blocks the model has actually used get retained.
- **token_merge** (new) — bipartite content-similarity merge on memory: every other block is paired with its neighbor, top-r most-similar pairs are merged, halving memory length.
- **recurrent_state** (new) — linear-attention with positive feature map ϕ and causal cumulative state $S_t = \sum_{s \le t} \phi(K_s) \otimes V_s$. Memory is $O(d_k^2)$ per layer, independent of T.

4 Correctness Suite

Each mechanism passes five CUDA tests before entering the sweep: shape/dtype, causality (future-position perturbations leave earlier outputs unchanged), mask correctness, gradient flow, and recorded peak memory. Current status: 55/55 green on one RTX 3090. Run via `python -m tests.test_forgetting_mechanisms`.

5 Setup

Table 1: Shared experimental setup for the scaling sweep.

Item	Value
Config preset	`CSAMinimumPaperConfig`
Parameters	18.35M–18.77M
Hidden size	256
Layers / heads	8 / 8
KV heads (GQA)	4
Context length sweep	{2048, 4096, 8192}
Batch size (scaled)	4 / 2 / 1 (constant tokens/step)
Optimizer	Muon (matrix) + AdamW (other)
Dataset	`processed_data/speedrun_40M`
Hardware	1×NVIDIA RTX 3090, 24GB
Budget per run	300 active training seconds
Eval cadence	every 200 steps
Seeds	1 (this run)

Fairness rule. Only the per-layer attention module changes between rows; model, data, optimizer, seed, and hardware are identical. Batch size scales with context length so tokens-per-step is constant at 8192, and each run is capped at 300 active training seconds.

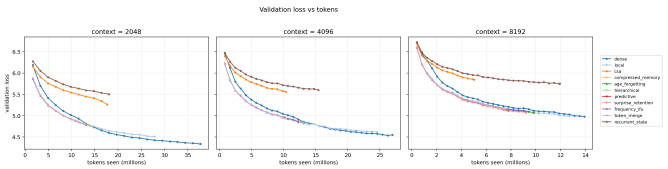
Metrics. Each run writes `metrics.json` with final validation loss, tokens seen, tok/s, active and wall time, peak allocated/reserved VRAM, and a per-eval loss history versus tokens and seconds.

6 Results

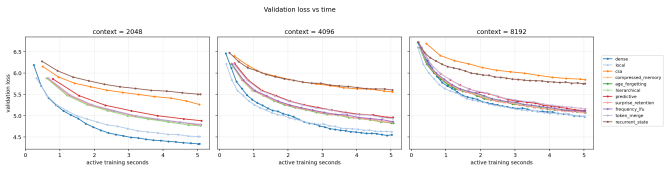
The full sweep completed: 33/33 runs finished successfully, with no OOMs or runtime failures. Table 2 gives the complete grid. The pattern is consistent across context lengths. Dense attention is the best validation-loss baseline and also the fastest path in this reference implementation. Local attention is the strongest cheaper baseline. The compressed-memory family clusters behind local, and CSA and recurrent state are substantially worse under this short training budget. Within the forgetting family, simple rules are the strongest: age decay and ungated compressed memory are near the top at 2K and 4K, while surprise retention is the best compressed-memory variant at 8K. The learned predictive router and token-merge variant are weaker here, which suggests that extra selection machinery can lose to simpler memory rules when the implementation is unfused and the run is short.

Table 2: Completed 33-run sweep. Each row is one 300-second run on the same RTX 3090. Final loss is not the only fair view because faster mechanisms process more tokens inside the time cap.

Context	Mechanism	Val loss	Tokens (M)	Tok/s	Time (s)	VRAM GiB
2048	age_forgetting	4.7595	13.9	45.7k	305	9.91
2048	compressed_memory	4.7677	14.0	45.8k	305	9.90
2048	csa	5.2627	17.7	58.3k	303	7.66
2048	dense	4.3386	37.7	124.1k	304	6.94
2048	frequency_lfu	4.7890	13.1	42.7k	306	9.91
2048	hierarchical	4.7589	13.9	45.4k	305	11.38
2048	local	4.5106	27.9	91.6k	304	7.00
2048	predictive	4.8816	10.9	35.4k	307	12.15
2048	recurrent_state	5.5019	18.0	59.1k	305	7.00
2048	surprise_retention	4.7906	13.4	43.7k	306	9.91
2048	token_merge	4.7969	12.9	42.3k	306	9.89
4096	age_forgetting	4.8268	13.1	43.1k	304	5.76
4096	compressed_memory	4.8275	13.1	43.2k	303	5.76
4096	csa	5.5567	10.3	34.1k	302	3.88
4096	dense	4.5522	26.9	89.4k	301	3.51
4096	frequency_lfu	4.8611	12.3	40.4k	303	5.76
4096	hierarchical	4.8367	13.1	43.2k	303	5.74
4096	local	4.6212	24.5	81.4k	301	3.57
4096	predictive	4.9611	10.1	33.3k	304	6.13
4096	recurrent_state	5.6000	15.4	50.9k	303	3.54
4096	surprise_retention	4.8577	12.7	41.9k	303	5.76
4096	token_merge	4.9331	10.4	34.3k	303	5.75
8192	age_forgetting	5.0689	9.8	32.5k	302	2.93
8192	compressed_memory	5.0733	9.7	32.3k	302	2.93
8192	csa	5.8409	5.0	16.7k	302	2.00
8192	dense	4.9768	13.9	46.2k	301	1.80
8192	frequency_lfu	5.1016	9.2	30.4k	302	2.93
8192	hierarchical	5.1066	9.0	29.9k	302	2.92
8192	local	4.9923	12.7	42.3k	301	1.86
8192	predictive	5.1282	7.8	25.8k	302	3.51
8192	recurrent_state	5.7510	11.9	39.6k	302	1.82
8192	surprise_retention	5.0660	9.2	30.6k	302	2.93
8192	token_merge	5.1591	7.5	24.9k	302	2.92

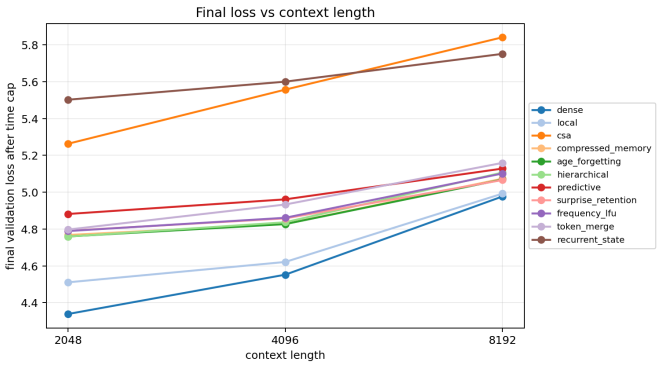


(a) Validation loss vs tokens seen. This is the fairest curve because the sweep is time-capped.

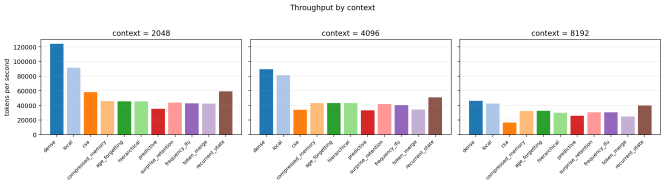


(b) Validation loss vs active training seconds.

Figure 2: Token-normalized and time-normalized views of the completed sweep. Dense and local dominate the reference implementations in this budget regime.



(a) Final validation loss by context length.



(b) Throughput by context length.

Figure 3: Cost side of the comparison. Longer contexts reduce throughput, so time-capped loss must be read together with tokens processed and wall time.

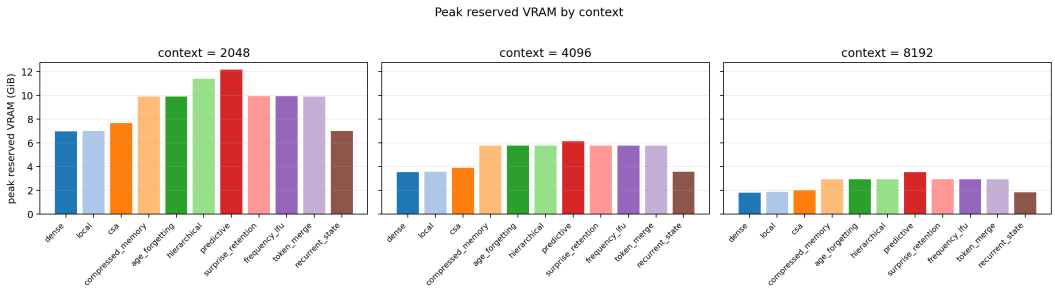


Figure 4: Peak reserved VRAM by context length. This is what tells us which mechanisms are likely to fit future kernels or multi-GPU scaling work.

Takeaway. The reference implementations do not show a quality win for the forgetting policies. Their value at this stage is diagnostic: the tested memory rules are correct and runnable, but the unfused compressed-memory path is too slow and too weak to beat the simple local baseline. A useful next paper should either improve the memory rule enough to beat local in loss-vs-tokens, or write a fused kernel and rerun this exact sweep.

7 Usefulness for Big Lab Researchers

The codebase is reference material for kernel and scaling work. Each policy is a plain-PyTorch module with math in the docstrings, paired with a passing causal correctness test. A kernel writer can port a policy to Triton or CUDA, A/B it against the reference for exactness, keep the same `metrics.json` schema, and rerun this sweep on larger GPUs. The training script is single-GPU first but the layout, config, checkpoints, and metrics files adapt directly to `torchrun`, DDP, or an internal harness.

8 Honest Limits

One seed, one 3090, one dataset, one parameter scale, unfused kernels. The numbers describe how each mechanism plateaus on this exact setup. The task is ordinary next-token prediction, which may not put enough pressure on the memory system to reveal when old information is useful. A fused-kernel rerun or a long-memory retrieval task could change the ranking.

9 Output Artifacts

The four newest mechanisms live in `models/new_forgetting.py`, the correctness suite in `tests/test_forgetting_mechanisms.py`, and the scaling sweep in `experiments/forgetting_scaling_sweep.py`.

10 Conclusion

The sweep turns ten memory ideas into runnable, causal PyTorch modules with passing correctness tests and a complete equal-time scaling table. Dense and local lead on this 18M reference run; among the memory family, age decay, ungated compressed memory, and surprise retention are the strongest candidates to fuse and rerun.

11 Next Steps

- Repeat the sweep with three seeds for dense, local, age_forgetting, surprise_retention, and compressed_memory.
- Write one fused kernel for the most promising compressed-memory path and compare it against the plain-PyTorch reference for exactness.
- Add a synthetic key-value retrieval task where the answer depends on old tokens. For example, place `key -> value` pairs thousands of tokens before a query and measure whether each mechanism retrieves the right value. This tests memory usefulness directly instead of only through language-model loss.
- If a mechanism beats local on loss-vs-tokens, graft that rule into the CSA lightning-indexer path and rerun the same table.

References

[1] DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024.

[2] DeepSeek-AI. *DeepSeek-V3 Technical Report*. 2024.

[3] V. Rosic. *Can Compressed Sparse Attention Help Under Equal GPU Time?* Internal research report, May 24, 2026.

[4] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. Le, and R. Salakhutdinov. *Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*. ACL 2019.

[5] J. W. Rae, A. Potapenko, S. J. Kayumov, and T. P. Lillicrap. *Compressive Transformers for Long-Range Sequence Modelling*. ICLR 2020.

[6] I. Beltagy, M. E. Peters, and A. Cohan. *Longformer: The Long-Document Transformer*. arXiv:2004.05150, 2020.

[7] M. Zaheer et al. *Big Bird: Transformers for Longer Sequences*. NeurIPS 2020.

[8] N. Kitaev, L. Kaiser, and A. Levskaya. *Reformer: The Efficient Transformer*. ICLR 2020.

[9] A. Roy, M. Saffar, A. Vaswani, and D. Grangier. *Efficient Content-Based Sparse Attention with Routing Transformers*. TACL 2021.

[10] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy. *Memorizing Transformers*. ICLR 2022.

[11] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis. *Generalization through Memorization: Nearest Neighbor Language Models*. ICLR 2020.

[12] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, J. Hoffman. *Token Merging: Your ViT But Faster*. ICLR 2023.

[13] A. Katharopoulos, A. Vyas, N. Pappas, F. Fleuret. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*. ICML 2020.